

Track 14 — Wer den Preis setzt, und was der Bestand bedeutet

Datum: 2026-08-28 **Ausgangsstand:** 93d2ea1 (Track 13), 1189 Tests **Endstand:** 1217 Tests (28 neu), Typecheck grün

Warum diese Strecke

Track 13 hat die Betrags-Manipulation im Checkout ausdrücklich als „ohne Befund“ abgehakt:

```
product_order [liest] die unit_price_cents der Positionen
```

Das stimmt für `/api/stripe/checkout` — die Route nimmt wirklich keinen Betrag aus dem Request entgegen. Nur endet die Kette dort nicht. `unit_price_cents` wird eine Ebene tiefer gebildet, in `createOrder`, aus einer Variante, die der Kunde selbst in seinen Warenkorb geschrieben hat. Die Prüfung hörte an der Stelle auf, an der der Wert *gelesen* wurde, statt dort weiterzugehen, wo er *entsteht*.

Der Shop war damit die am wenigsten geprüfte Strecke der Anwendung: die vier Routen `/api/cart`, `/api/orders`, `/api/provider/products` und `/api/provider/products/[id]` hatten zusammen kein einziges Schema.

Befunde

1 — P0: Der Stückpreis war frei wählbar

Die Kette, Glied für Glied:

1. `POST /api/cart` nahm `variantId` roh aus dem Body. 2. `addToCart` schrieb ihn ungeprüft nach `cart_items.variant_id`. 3. `getCartItems` bettet `product_variants` über genau diesen Fremdschlüssel ein. Eine Einbettung folgt der Spalte, nicht dem Produkt — sie liefert also auch eine Variante, die zu einem völlig anderen Produkt gehört. 4. `createOrder` nahm `variant.price_cents` als Stückpreis:

```
`ts const unitPrice = variant?.price_cents || product.price_cents`
```

5. `/api/stripe/checkout` baut daraus `unit_amount` der Line-Items.

Ein einziger Request genügte:

```
POST /api/cart
{ "productId": "<teures Produkt>", "variantId": "<Variante eines billigen Produkts>" }
```

Stripe zog daraufhin den Betrag der fremden Variante ein, der Webhook setzte die Bestellung auf `paid / confirmed`, und der Verkäufer sah eine reguläre, vollständig bezahlte Bestellung. Die Ware

ging zum Preis eines beliebigen anderen Katalogartikels raus. Der Angreifer brauchte nichts weiter als ein Kundenkonto und zwei öffentlich sichtbare Produkt-IDs.

Jetzt: die Variante wird an zwei Stellen gegen ihr Produkt gehalten — beim Hinzufügen (`variant.product_id === product.id`) und noch einmal beim Bestellen, damit auch bereits gespeicherte Zeilen aus der Zeit vor dem Fix nicht durchkommen. Zusätzlich muss die Variante aktiv sein.

2 — P1: Die Gratis-Variante kostete den vollen Produktpreis

`variant?.price_cents || product.price_cents` — dasselbe `||` in die andere Richtung. Eine Variante mit 0 Cent ist ein Preis, kein fehlender Wert; mit `||` fiel sie auf den vollen Produktpreis zurück. Der Kunde zahlte für das Gratis-Muster. Derselbe Ausdruck stand dreimal im Frontend (`CartProvider`, `CartDrawer`, `ProductDetailClient`), die angezeigte Summe war also konsistent falsch — und stimmte mit dem überein, was Stripe einzog.

Jetzt: `??` an allen vier Stellen.

3 — P1: Die Menge war unvalidiert

`quantity || 1`. Durchgelassen wurden:

Eingabe	Ergebnis vorher
<code>-5</code>	negativer Positionsbetrag, negative Zwischensumme
<code>0.5</code>	Stripe lehnt die Session ab — die Bestellung steht aber schon
<code>1e9</code>	Bestellung über einen willkürlichen Betrag
<code>"1"</code> (String)	<code>existing.quantity + "1"</code> → aus <code>1 + "1"</code> wurde die Menge 11

Jetzt: ganze Zahl, 1 bis 99 je Position, höchstens 50 Positionen je Warenkorb — im Schema der Route und noch einmal im Service, weil `createOrder` auch über bereits gespeicherte Zeilen läuft.

4 — P1: Der Bestand war Zierde

`stock_quantity` wurde **nirgends** geprüft und **nirgends** abgezogen. Ein Produkt mit 0 Stück war unbegrenzt verkäuflich; zehn Kunden konnten dasselbe letzte Stück bezahlen. Die Produktseite zeigte „Ausverkauft“ korrekt an — serverseitig hatte die Zahl keinerlei Wirkung.

Dazu ein Fehler beim Anlegen:

```
is_unlimited_stock: !stockQuantity
```

Wer `stockQuantity: 0` angab, also „ausverkauft“, bekam „unbegrenzt lieferbar“.

Jetzt: geprüft wird beim Hinzufügen, beim Ändern der Menge und beim Bestellen. Gebucht wird der Bestand dort, wo das Geld ankommt — im Stripe-Webhook, atomar je Position über Compare-and-Swap (`.eq('stock_quantity', gelesen)`). Reicht er zwischen Bestellung und Zahlung nicht mehr, geht die Zahlung vollständig zurück und die Bestellung wird storniert; das ist dieselbe Linie wie die Overlap-Defense der Miete. Eine Erstattung gibt den Bestand wieder frei, und zwar genau einmal — `charge.refunded` wird von Stripe mehrfach zugestellt.

Beim Anlegen gilt jetzt: keine Angabe heißt „kein Bestand geführt“ (das Dashboard-Formular hat kein Bestandsfeld, dort angelegte Produkte bleiben verkäuflich), eine ausdrückliche `0` heißt ausverkauft.

5 — P1: Die Bestellung konnte ohne Positionen entstehen

```
await supabase.from('order_items').insert(items) // Fehler nie geprüft
await supabase.from('cart_items').delete().eq('customer_id', customerId)
```

Schlug der Insert fehl, blieb eine Bestellung **mit** Gesamtbetrag und **ohne** Positionen stehen. Der Checkout baute daraus eine Stripe-Session, die nur den Versand enthielt — und der Warenkorb des Kunden war trotzdem weg. Ebenso still: `if (!product) continue` ließ einen inzwischen gelöschten Artikel wortlos aus der Bestellung fallen.

Jetzt: Fehler wird geprüft, die Bestellung im Fehlerfall wieder entfernt, der Warenkorb erst nach vollständigem Erfolg geleert, und ein nicht mehr lieferbarer Artikel wird beim Namen genannt statt verschluckt.

6 — P1: Ausgelistete Produkte blieben bestellbar

`DELETE /api/provider/products/[id]` setzt `is_active: false`. Weder `addToCart` noch `createOrder` haben das je gelesen. Das Produkt verschwand aus dem Katalog und wurde weiter verkauft.

7 — P1: Öffnungszeiten aus dem Dashboard waren unsichtbar

`salons.opening_hours` wird an drei Stellen gelesen — `/api/availability`, `lib/scheduling.ts` und der Schema.org-Export — und alle drei erwarten deutsche Tageskürzel (`{ "Mo": "09:00 - 18:00" }`). Geschrieben wurde die Spalte an zwei Stellen in zwei Formaten: `/anbieter/mein-salon/zeiten` schrieb über `/api/me/salon` die Kürzel, das Anbieter-Dashboard über `/api/provider/salon` die ausgeschriebenen Tagesnamen (`"Montag"`).

Kein Leser kennt `"Montag"`. Wer seine Zeiten im Dashboard pflegte, sah sie gespeichert — `/api/availability` fand für jeden Tag nichts und antwortete mit `{ slots: [] }`. Der Salon war nicht buchbar. Und das Speichern im Dashboard **überschrieb** ein zuvor korrekt gepflegtes Objekt.

Der Kommentar in `/api/me/salon` hatte genau davor gewarnt („Ein zweites Format hätte die Buchungslogik still ausgehebelt“) — das zweite Format existierte zu dem Zeitpunkt schon.

Jetzt: das Format steht einmal in `src/lib/opening-hours.ts`, beide Routen validieren dagegen, das Dashboard-Formular schreibt die Kürzel, und Bestandsdaten im Langformat werden beim Anzeigen umgerechnet, statt zu verschwinden.

8 — P2: `PATCH /api/provider/salon` ohne Schema (offener Punkt aus Track 13)

```
for (const key of allowed) if (key in body) updates[key] = body[key]
```

Die Allowlist entschied nur, welche Spalte beschrieben wird, nicht womit. `name: {}`, `postal_code: 12345` als Zahl oder eine 2-MB- `description` gingen unverändert in die Tabelle, aus der die öffentliche Salon-Seite und der Schema.org-Export lesen.

9 — P2: Anbieter mit zwei Salons waren ausgesperrt

`.single()` auf `salons` — bei zwei Zeilen antwortet PostgREST mit PGRST116, und der Inhaber bekam „Kein Salon gefunden“. Betroffen waren `/api/provider/salon` (PATCH) und alle vier Handler unter `/api/provider/products`. Dasselbe Muster war in `/api/provider/services` schon behoben; jetzt laufen alle über `getOwnedSalon`.

10 — P2: Fremd-ID meldete Erfolg

`DELETE /api/provider/products/[id]` antwortete `success: true`, auch wenn die ID zu einem fremden Salon gehörte und nichts getroffen wurde. Der Besitz war über `.eq('salon_id', ...)` korrekt geschützt — die Antwort log trotzdem.

11 — P2: Die Produktseite verwarf die Antwort

`handleAddToCart` wertete nur 401 aus. Da die Route jetzt wirklich ablehnt (ausverkauft, ausgelistet, Menge), hätte der Knopf sonst weiter Erfolg gemeldet, während im Warenkorb nichts liegt. Fehlermeldung ergänzt.

Was geprüft wurde und in Ordnung war

- **Stripe-Webhook-Signatur.** `constructEvent` über den Rohtext; fehlender Header und fehlendes Secret werden getrennt abgewiesen. Ein Replay ist wirkungslos, weil jeder Zweig idempotent ist (Doppelzahlungs-Guard, CAS-Claim) — der Bestandsabzug ist jetzt Teil derselben Idempotenz.
- **Provider-Isolation.** Alle schreibenden Provider-Routen lösen den Salon über die eigene `owner_id` auf; keine nimmt eine Salon-ID aus dem Request. `products` wird zusätzlich über `.eq('salon_id', ...)` eingegrenzt.
- **Miet-Bestellstrecke.** `POST /api/rental-bookings` nimmt nur `equipmentId` und Zeitraum, rechnet serverseitig, prüft Overlap und fängt `23P01` ab. `rental-equipment` validiert vollständig über Zod.
- `GET /api/orders/[id]` ist über `customer_id` eingegrenzt, `PATCH` ist auf `admin/super_admin` beschränkt.
- `/api/setup/promote-admin` — Rate-Limit, zeitkonstanter Vergleich, Mindestschlüssellänge.

Offen

- **`salons.is_active` / `is_verified` auf der Mietstrecke** — unverändert offen aus Track 13. Frisch registrierte Anbieter sind sofort im Markt sichtbar. Das Nachziehen ist eine Produktentscheidung, keine Fehlerbehebung.
- **`passwordMustChange`** wird ausgewertet, aber nie gesetzt.
- `PATCH /api/orders/[id]` schreibt `status` als freien String; welche Werte die Spalte live zulässt, ist von hier aus nicht prüfbar (kein DB-Zugang für Agents).
- **Bestellungen erzeugen keine Provision und keine Anbieter-Benachrichtigung.** Anders als Termin und Miete bucht der Shop keine `platform_transactions`; der Verkäufer erfährt von der bezahlten Bestellung nur über die Liste.

Zahlen

Tests vorher	1189
Tests nachher	1217
Neue Tests	28
Geänderte Produktivdateien	11
Neue Module	<code>src/lib/opening-hours.ts</code>
Migrationen	keine — alle berührten Spalten existieren live (Sonde 2026-08-28)